

WEEK8-SKILL

Task1:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
UW PICO 5.09 File: combined_redirect.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char filename[100];

    printf("Enter output file name: ");
    scanf("%s", filename);

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("File");
        return 1;
    }

    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        dup2(fd, STDOUT_FILENO);
        dup2(fd, STDERR_FILENO);
        close(fd);

        execlp("sh", "sh", "-c",
              "echo This is normal output; echo This is error output >&2",
              NULL);

        exit(1);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("Combined redirection completed\n");

    return 0;
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

```
vishalkarnati@VISHALS-MacBook-Air Desktop % cd os
vishalkarnati@VISHALS-MacBook-Air os % mkdir week8skill
zsh: command not found: mkdir
vishalkarnati@VISHALS-MacBook-Air os % mkdir week8skill
vishalkarnati@VISHALS-MacBook-Air os % cd week8skill
vishalkarnati@VISHALS-MacBook-Air week8skill % nano combined_redirect.c
vishalkarnati@VISHALS-MacBook-Air week8skill % gcc combined_redirect.c -o combined_redirect
vishalkarnati@VISHALS-MacBook-Air week8skill % ./combined_redirect
Enter output file name: output.txt
Combined redirection completed
vishalkarnati@VISHALS-MacBook-Air week8skill % cat output.txt
This is normal output
This is error output
vishalkarnati@VISHALS-MacBook-Air week8skill %
```

Task2:

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
UW PICO 5.09 File: complex.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t pid1 = fork();
    if (pid1 == -1)
    {
        perror("fork");
        return 1;
    }
    if (pid1 == 0)
    {
        int input_fd = open("input.txt", O_RDONLY);
        if (input_fd == -1)
        {
            perror("input.txt");
            exit(1);
        }

        dup2(input_fd, STDIN_FILENO);
        close(input_fd);
        close(pipefd[0]);
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);
        execlp("cat", "cat", NULL);

        perror("cat");
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == -1)
    {
        perror("fork");
        return 1;
    }
    if (pid2 == 0)
    {
        int output_fd = open("output.txt",
                             O_WRONLY | O_CREAT | O_TRUNC,
                             0644);
        if (output_fd == -1)
        {
            perror("output.txt");
            exit(1);
        }

        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        dup2(output_fd, STDOUT_FILENO);
        close(output_fd);
        execlp("grep", "grep", "Hello", NULL);

        perror("grep");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);
}
```

```
UW PICO 5.09 File: complex.c Modified
{
    perror("input.txt");
    exit(1);
}

dup2(input_fd, STDIN_FILENO);
close(input_fd);
close(pipefd[0]);
dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[1]);
execlp("cat", "cat", NULL);

perror("cat");
exit(1);
}

pid_t pid2 = fork();
if (pid2 == -1)
{
    perror("fork");
    return 1;
}
if (pid2 == 0)
{
    int output_fd = open("output.txt",
                         O_WRONLY | O_CREAT | O_TRUNC,
                         0644);
    if (output_fd == -1)
    {
        perror("output.txt");
        exit(1);
    }

    close(pipefd[1]);
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);
    dup2(output_fd, STDOUT_FILENO);
    close(output_fd);
    execlp("grep", "grep", "Hello", NULL);

    perror("grep");
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);
}
```

```
UW PICO 5.09 File: complex.c Modified
{
    int output_fd = open("output.txt",
                        O_WRONLY | O_CREAT | O_TRUNC,
                        0644);

    if (output_fd == -1)
    {
        perror("output.txt");
        exit(1);
    }

    close(pipefd[1]);
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[0]);
    dup2(output_fd, STDOUT_FILENO);
    close(output_fd);
    execlp("grep", "grep", "Hello", NULL);

    perror("grep");
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("Complex command completed\n");

return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: input.txt Modified
Hello
Operating Systems
Hello World
Linux
Good Morning
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
vishalkarnati@VISHALS-MacBook-Air week8skill % nano complex.c
vishalkarnati@VISHALS-MacBook-Air week8skill % nano input.txt
vishalkarnati@VISHALS-MacBook-Air week8skill % gcc complex.c -o complex
vishalkarnati@VISHALS-MacBook-Air week8skill % ./complex
Complex command completed
vishalkarnati@VISHALS-MacBook-Air week8skill % cat output.txt
Hello
Hello World
vishalkarnati@VISHALS-MacBook-Air week8skill %
```

Task3:

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
UW PICO 5.09 File: background.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        int background = 0;

        int length = strlen(command);

        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';
        }

        if (strcmp(command, "sleep") == 0)
        {
            printf("Usage: sleep <seconds> [&]\n");
            continue;
        }

        char *args[10];
        int count = 0;

        char *token = strtok(command, " ");

        while (token != NULL && count < 9)
        {
            args[count] = token;
            count++;

            token = strtok(NULL, " ");
        }

        args[count] = NULL;

        if (count == 0)
        {
            continue;
        }

        Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: background.c Modified
        background = 1;
        command[length - 1] = '\0';
    }

    if (strcmp(command, "sleep") == 0)
    {
        printf("Usage: sleep <seconds> [&]\n");
        continue;
    }

    char *args[10];
    int count = 0;

    char *token = strtok(command, " ");

    while (token != NULL && count < 9)
    {
        args[count] = token;
        count++;

        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    if (count == 0)
    {
        continue;
    }

    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    if (background)
    {
        printf("[Background job] PID = %d\n", pid);
    }
    else
    {
        waitpid(pid, NULL, 0);
    }
}

return 0;

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
~/Desktop/os/week8skill - sleep - background  ~ -- -zsh +
vishalkarnati@VISHALS-MacBook-Air week8skill % ./background
mysHELL> sleep 5
mysHELL> sleep 10 &
[background job] PID = 4138
mysHELL> 
```

```
~/Desktop/os/week8skill - sleep - background  ~ -- -zsh +
Last login: Thu Sep 17 21:50:07 on ttys000
(eval):8: command not found: >
vishalkarnati@VISHALS-MacBook-Air ~ % ps
  PID TTY          TIME CMD
 3166 ttys000    0:00.38 -zsh
  4133 ttys000    0:00.01 ./background
  4138 ttys000    0:00.00 <defunct>
  4140 ttys001    0:00.03 -zsh
vishalkarnati@VISHALS-MacBook-Air ~ % 
```

Task4:

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
week8skill - nano jobs.c - 204x64
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    int running;
};

struct Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char command[])
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    jobs[job_count].running = 1;

    printf("[Job] %d Running %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
    next_job_id++;
}

void check_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );
        }
    }
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

```

week8skill - nano jobs.c - 204x64

jobs[job_count].pid = pid;
strcpy(jobs[job_count].command, command);

jobs[job_count].running = 1;

printf("[%d] %d Running %s\n",
       jobs[job_count].job_id,
       jobs[job_count].pid,
       jobs[job_count].command);

job_count++;
next_job_id++;
}

void check_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result > 0)
            {
                jobs[i].running = 0;

                printf("[%d] Done %s\n",
                       jobs[i].job_id,
                       jobs[i].command);
            }
        }
    }
}

void display_jobs()
{
    check_jobs();

    if (job_count == 0)
    {
        printf("No jobs\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] %d Running %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
    }
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
 Exit Justify Where is Next Pg UnCut Text To Spell

```

week8skill - nano jobs.c - 204x64

    {
        jobs[i].running = 0;

        printf("[%d] Done %s\n",
               jobs[i].job_id,
               jobs[i].command);
    }
}

void display_jobs()
{
    check_jobs();

    if (job_count == 0)
    {
        printf("No jobs\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] %d Running %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
    }
}

void remove_completed_jobs()
{
    int new_count = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            jobs[new_count] = jobs[i];
            new_count++;
        }
    }

    job_count = new_count;
}

int main()
{
    char command[100];

    while (1)
    {
        check_jobs();

        printf("myshell> ");
        fflush(stdout);
    }
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
 Exit Justify Where is Next Pg UnCut Text To Spell

```
week8skill - nano jobs.c - 204x64
}
}
void remove_completed_jobs()
{
    int new_count = 0;
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            jobs[new_count] = jobs[i];
            new_count++;
        }
    }
    job_count = new_count;
}
int main()
{
    char command[100];
    while (1)
    {
        check_jobs();

        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strlen(command) - 1] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "jobs") == 0)
        {
            display_jobs();
            remove_completed_jobs();
            continue;
        }

        int background = 0;
        int length = strlen(command);

        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';

            while (length > 0 && command[length - 2] == ' ')
            {
                command[length - 2] = '\0';
                length--;
            }
        }
    }
}
Get Help      WriteOut      Read File      Prev Pg      Cut Text      Cur Pos
Exit          Justify        Where is       Next Pg      UnCut Text    To Spell
```

```
UW PICO 5.09      File: jobs.c      Modified
fgets(command, sizeof(command), stdin);
command[strlen(command) - 1] = '\0';

if (strcmp(command, "exit") == 0)
{
    break;
}

if (strcmp(command, "jobs") == 0)
{
    display_jobs();
    remove_completed_jobs();
    continue;
}

int background = 0;
int length = strlen(command);

if (length > 0 && command[length - 1] == '&')
{
    background = 1;
    command[length - 1] = '\0';

    while (length > 0 && command[length - 2] == ' ')
    {
        command[length - 2] = '\0';
        length--;
    }
}

char original_command[100];
strcpy(original_command, command);

char *args[10];
int count = 0;

char *token = strtok(command, " ");

while (token != NULL && count < 9)
{
    args[count] = token;
    count++;

    token = strtok(NULL, " ");
}

args[count] = NULL;

if (count == 0)
{
    continue;
}

pid_t pid = fork();

if (pid == -1)
```

```
UW PICO 5.09 File: jobs.c Modified
}
char original_command[100];
strcpy(original_command, command);
char *args[10];
int count = 0;
char *token = strtok(command, " ");
while (token != NULL && count < 9)
{
    args[count] = token;
    count++;
    token = strtok(NULL, " ");
}
args[count] = NULL;
if (count == 0)
{
    continue;
}
pid_t pid = fork();
if (pid == -1)
{
    perror("fork");
    continue;
}
if (pid == 0)
{
    execvp(args[0], args);
    perror("exec");
    exit(1);
}
if (background)
{
    add_job(pid, original_command);
}
else
{
    waitpid(pid, NULL, 0);
}
}
return 0;
}
Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week8skill % nano jobs.c
vishalkarnati@VISHALS-MacBook-Air week8skill % gcc jobs.c -o jobs
vishalkarnati@VISHALS-MacBook-Air week8skill % ./jobs
myshell> sleep 10 &
[1] 4312 Running sleep 10
myshell> sleep 20 &
[2] 4316 Running sleep 20
myshell> jobs
[1] Done sleep 10
[2] 4316 Running sleep 20
myshell> jobs
[2] Done sleep 20
myshell>
```